LIBRARY MANAGEMENT SYSTEM

Aim

To develop a simple library website where students can search books, check borrow history and fines and download study material, and the librarian can manage books, issue and return books and calculate fines.

Algorithm

1. Start and open the library website.
2. Login or Register. The system checks the details and opens the Admin or Student page.
3. Admin adds, updates or deletes books and uploads study material (year and semester wise).
4. User searches a book by title, author or ISBN and views its details.
5. Admin issues a book to a student and saves the issue date and expiry date.
6. Admin returns the book. If it is late, fine = late days x fine per day.
7. Student views borrow history, fine and study material, then logs out. Stop.

CODE:

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Campus Library Management</title>
  <link rel="stylesheet" href="library.css">
</head>
<body>

<header>
  <h1>📚 Campus Library</h1>
  <nav>
    <a href="#home">Home</a>
    <a href="#books">Books</a>
    <a href="#materials">Materials</a>
    <a href="#account">My Account</a>
    <a href="#admin">Admin</a>
  </nav>
</header>

<section class="hero" id="home">
  <h2>Your Gateway to Knowledge</h2>
  <p>Search books, manage borrowing and access study materials.</p>
  <a href="#books" class="hero-btn">Explore Books</a>
</section>
```

```
<section id="account">
  <h2>Student Registration / Login</h2>
  <div class="form-box">
    <input id="studentName" placeholder="Full Name">
    <input id="studentEmail" type="email" placeholder="Email Address">
    <input id="studentPassword" type="password" placeholder="Password">
    <button onclick="registerStudent()">Register</button>
    <button onclick="loginStudent()">Login</button>
    <p id="accountMessage"></p>
  </div>

  <div class="form-box">
    <h3>My Profile</h3>
    <div id="profileDetails">Please log in to view your profile.</div>
    <button onclick="logoutStudent()">Logout</button>
  </div>
</section>

<section id="books">
  <h2>Available Books</h2>

  <div class="search-bar">
    <input id="bookSearch" placeholder="Search by book title or author..."
        oninput="displayBooks()">
    <select id="bookCategory" onchange="displayBooks()">
      <option value="All">All Categories</option>
      <option value="Programming">Programming</option>
      <option value="Database">Database</option>
      <option value="Artificial Intelligence">Artificial Intelligence</option>
      <option value="Mathematics">Mathematics</option>
    </select>
  </div>

  <div id="bookList" class="book-grid"></div>
</section>

<section id="history">
  <h2>My Borrowing History</h2>
  <div id="historyList"></div>
</section>

<section id="fine">
  <h2>My Fine Details</h2>
  <div id="fineDetails" class="form-box">
    Log in to check your fine.
  </div>
</section>

<section id="materials">
  <h2>Study Materials</h2>
```

```
    <div class="search-bar">
      <select id="materialYear" onchange="displayMaterials()">
        <option value="All">All Years</option>
        <option value="1">First Year</option>
        <option value="2">Second Year</option>
        <option value="3">Third Year</option>
        <option value="4">Fourth Year</option>
      </select>

      <select id="materialSemester" onchange="displayMaterials()">
        <option value="All">All Semesters</option>
        <option value="1">Semester 1</option>
        <option value="2">Semester 2</option>
        <option value="3">Semester 3</option>
        <option value="4">Semester 4</option>
        <option value="5">Semester 5</option>
        <option value="6">Semester 6</option>
        <option value="7">Semester 7</option>
        <option value="8">Semester 8</option>
      </select>
    </div>

    <div id="materialList" class="material-grid"></div>
  </section>

  <section id="contact">
    <h2>Contact the Library</h2>
    <div class="form-box">
      <input id="contactName" placeholder="Your Name">
      <input id="contactEmail" type="email" placeholder="Your Email">
      <textarea id="contactMessage" placeholder="Enter your message"></textarea>
      <button onclick="sendMessage()">Send Message</button>
      <p id="contactResult"></p>
    </div>
  </section>

  <section id="admin">
    <h2>Librarian Admin Dashboard</h2>

    <div class="form-box">
      <h3>Admin Login</h3>
      <input id="adminEmail" type="email" placeholder="Admin Email">
      <input id="adminPassword" type="password" placeholder="Admin Password">
      <button onclick="adminLogin()">Login as Librarian</button>
      <p id="adminMessage"></p>
    </div>

    <div id="adminDashboard" style="display:none">
      <div class="form-box">
```

```
        <h3>Add New Book</h3>
        <input id="newBookTitle" placeholder="Book Title">
        <input id="newBookAuthor" placeholder="Author">
        <input id="newBookCategory" placeholder="Category">
        <input id="newBookCopies" type="number" min="1" placeholder="Number of
Copies">
        <textarea id="newBookDescription" placeholder="Book Description"></textarea>
        <button onclick="addBook()">Add Book</button>
    </div>

    <div class="form-box">
        <h3>Issue a Book</h3>
        <input id="issueEmail" type="email" placeholder="Student Email">
        <select id="issueBookId"></select>
        <button onclick="issueBook()">Issue Book</button>
    </div>

    <div class="form-box">
        <h3>Manage Books and Returns</h3>
        <div id="adminBookList"></div>
    </div>

    <div class="form-box">
        <h3>Manage Borrowing and Fines</h3>
        <div id="adminBorrowList"></div>
    </div>

    <div class="form-box">
        <h3>Upload Study Material</h3>
        <input id="materialTitle" placeholder="Material Title">
        <input id="materialSubject" placeholder="Subject">
        <select id="uploadYear">
            <option value="1">First Year</option>
            <option value="2">Second Year</option>
            <option value="3">Third Year</option>
            <option value="4">Fourth Year</option>
        </select>
        <select id="uploadSemester">
            <option value="1">Semester 1</option>
            <option value="2">Semester 2</option>
            <option value="3">Semester 3</option>
            <option value="4">Semester 4</option>
            <option value="5">Semester 5</option>
            <option value="6">Semester 6</option>
            <option value="7">Semester 7</option>
            <option value="8">Semester 8</option>
        </select>
        <label>Select a PDF file:</label>
        <input id="materialFile" type="file" accept=".pdf">
        <button onclick="uploadMaterial()">Add Study Material</button>
```

```
      </div>

      <div class="form-box">
        <h3>Manage Study Materials</h3>
        <div id="adminMaterialList"></div>
      </div>

      <div class="form-box">
        <h3>Registered Students</h3>
        <div id="adminStudentList"></div>
      </div>

      <div class="form-box">
        <h3>Contact Messages</h3>
        <div id="adminMessageList"></div>
      </div>
    </div>
</section>

<footer>
  <p>© 2026 Campus Library Management System</p>
</footer>

<script src="library.js"></script>
</body>
</html>
```

CSS:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: Arial, sans-serif;
  scroll-behavior: smooth;
}

body {
  background: #f4f7fc;
  color: #26344a;
}

header {
  background: #253b80;
  color: white;
  padding: 18px 5%;
  display: flex;
  justify-content: space-between;
```

```
    align-items: center;
    gap: 15px;
    flex-wrap: wrap;
}

header h1 {
    font-size: 26px;
}

nav {
    display: flex;
    flex-wrap: wrap;
    gap: 16px;
}

nav a {
    text-decoration: none;
    color: white;
    font-weight: bold;
}

nav a:hover {
    color: #aee8ff;
}

button, .hero-btn {
    background: #3977db;
    color: white;
    padding: 11px 17px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    text-decoration: none;
    display: inline-block;
    margin: 5px 3px;
}

button:hover, .hero-btn:hover {
    background: #2457ac;
}

.hero {
    text-align: center;
    padding: 65px 20px;
    background: linear-gradient(120deg, #dbeafe, #d5f5f6);
}
```

```
.hero h2 {
   font-size: 36px;
   color: #253b80;
   margin-bottom: 15px;
}

.hero p {
   margin-bottom: 25px;
   font-size: 17px;
}

section {
   padding: 35px 6%;
}

section > h2 {
   text-align: center;
   color: #253b80;
   margin-bottom: 25px;
}

.form-box {
   max-width: 700px;
   margin: 15px auto;
   background: white;
   padding: 22px;
   border-radius: 10px;
   box-shadow: 0 3px 12px #253b8012;
}

.form-box h3 {
   color: #253b80;
   margin-bottom: 12px;
}

input, select, textarea {
   width: 100%;
   padding: 12px;
   margin: 8px 0;
   border: 1px solid #ccd7e8;
   border-radius: 6px;
   font-size: 15px;
   background: #fff;
}

textarea {
   min-height: 85px;
   resize: vertical;
```

```
}

.search-bar {
  display: grid;
  grid-template-columns: 2fr 1fr;
  gap: 12px;
  max-width: 1000px;
  margin: 0 auto 25px;
}

.book-grid, .material-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}

.book-card, .material-card {
  background: white;
  padding: 22px;
  border-radius: 10px;
  box-shadow: 0 3px 12px #253b8015;
  border-top: 4px solid #5b8def;
}

.book-card h3, .material-card h3 {
  color: #253b80;
  margin-bottom: 12px;
}

.book-card p, .material-card p {
  margin: 8px 0;
  line-height: 1.5;
}

.available {
  color: #168455;
  font-weight: bold;
}

.unavailable {
  color: #c0394b;
  font-weight: bold;
}

#history, #materials {
  background: #eaf1ff;
}
```

```
.history-card, .admin-card, .message-card {
   background: white;
   max-width: 900px;
   margin: 12px auto;
   padding: 18px;
   border-radius: 8px;
   box-shadow: 0 2px 9px #0000000c;
}

.history-card p, .admin-card p {
   margin: 7px 0;
   overflow-wrap: anywhere;
}

.status {
   color: #147d64;
   font-weight: bold;
}

#admin {
   background: #edf7f6;
}

#adminDashboard {
   max-width: 1100px;
   margin: auto;
}

#adminDashboard .form-box {
   max-width: 900px;
}

footer {
   text-align: center;
   padding: 22px;
   background: #253b80;
   color: white;
}

@media (max-width: 650px) {
   header {
      flex-direction: column;
   }

   nav {
      justify-content: center;
```

```
    }

    .hero h2 {
        font-size: 28px;
    }

    .search-bar {
        grid-template-columns: 1fr;
    }

    section {
        padding: 25px 5%;
    }
}
```

JAVASCRIPT:

```
// Sample library data
const defaultBooks = [
    {
        id: 1,
        title: "Java Programming",
        author: "Herbert Schildt",
        category: "Programming",
        copies: 4,
        description: "Learn Java programming and object-oriented concepts."
    },
    {
        id: 2,
        title: "Python Basics",
        author: "Eric Matthes",
        category: "Programming",
        copies: 3,
        description: "A beginner-friendly introduction to Python."
    },
    {
        id: 3,
        title: "Database Systems",
        author: "Raghu Ramakrishnan",
        category: "Database",
        copies: 2,
        description: "Learn database design, SQL and data management."
    },
    {
        id: 4,
        title: "Artificial Intelligence",
        author: "Stuart Russell",
        category: "Artificial Intelligence",
```

```
        copies: 2,
        description: "Introduction to intelligent systems and AI methods."
    },
    {
        id: 5,
        title: "Engineering Mathematics",
        author: "B. S. Grewal",
        category: "Mathematics",
        copies: 5,
        description: "Mathematical methods for engineering students."
    }
];

function loadData(key, fallback) {
    try {
        const value = localStorage.getItem(key);
        return value === null ? fallback : JSON.parse(value);
    } catch (error) {
        return fallback;
    }
}

let books = loadData("libraryBooks", defaultBooks);
let students = loadData("libraryStudents", []);
let borrowRecords = loadData("libraryBorrowRecords", []);
let materials = loadData("libraryMaterials", []);
let messages = loadData("libraryMessages", []);

let currentStudent = null;
let isAdmin = false;

function saveData() {
    localStorage.setItem("libraryBooks", JSON.stringify(books));
    localStorage.setItem("libraryStudents", JSON.stringify(students));
    localStorage.setItem("libraryBorrowRecords", JSON.stringify(borrowRecords));
    localStorage.setItem("libraryMaterials", JSON.stringify(materials));
    localStorage.setItem("libraryMessages", JSON.stringify(messages));
}

function safeText(value) {
    return String(value ?? "").replace(/[&<>"']/g, character => ({
        "&": "&amp;",
        "<": "&lt;",
        ">": "&gt;",
        "\"": "&quot;",
        "'": "&#39;"
    })[character]);
}
```

```
function makeId() {
  return Date.now() + Math.floor(Math.random() * 1000);
}

// Student registration
function registerStudent() {
  const name = document.getElementById("studentName").value.trim();
  const email = document.getElementById("studentEmail").value.trim().toLowerCase();
  const password = document.getElementById("studentPassword").value;

  if (!name || !email.includes("@") || password.length < 4) {
    alert("Enter your name, valid email and password (at least 4 characters).");
    return;
  }

  if (students.some(student => student.email === email)) {
    alert("This email is already registered. Please log in.");
    return;
  }

  students.push({ id: makeId(), name, email, password });

  saveData();
  loginStudent();
}

// Student login
function loginStudent() {
  const email = document.getElementById("studentEmail").value.trim().toLowerCase();
  const password = document.getElementById("studentPassword").value;

  const student = students.find(
    item => item.email === email && item.password === password
  );

  if (!student) {
    document.getElementById("accountMessage").textContent =
      "Invalid login. Register first if you are a new student.";
    return;
  }

  currentStudent = student;

  document.getElementById("accountMessage").textContent =
    "Welcome, " + student.name + "! Login successful.";
```

```
    displayProfile();
    displayHistory();
    displayFine();
}

function logoutStudent() {
    currentStudent = null;

    document.getElementById("accountMessage").textContent = "You are logged out.";
    document.getElementById("profileDetails").textContent =
        "Please log in to view your profile.";

    displayHistory();
    displayFine();
}

function displayProfile() {
    if (!currentStudent) return;

    document.getElementById("profileDetails").innerHTML = `
        <p><strong>Name:</strong> ${safeText(currentStudent.name)}</p>
        <p><strong>Email:</strong> ${safeText(currentStudent.email)}</p>
        <p><strong>Student ID:</strong> ${currentStudent.id}</p>
    `;
}

// Book searching
function displayBooks() {
    const search = document.getElementById("bookSearch").value.toLowerCase();
    const category = document.getElementById("bookCategory").value;

    const filteredBooks = books.filter(book => {
        const matchesSearch =
            book.title.toLowerCase().includes(search) ||
            book.author.toLowerCase().includes(search) ||
            book.description.toLowerCase().includes(search);

        const matchesCategory =
            category === "All" || book.category.toLowerCase() === category.toLowerCase();

        return matchesSearch && matchesCategory;
    });

    const list = document.getElementById("bookList");
```

```
    if (filteredBooks.length === 0) {
        list.innerHTML = "<p>No books found.</p>";
        return;
    }

    list.innerHTML = filteredBooks.map(book => `
        <div class="book-card">
            <h3>${safeText(book.title)}</h3>
            <p><strong>Author:</strong> ${safeText(book.author)}</p>
            <p><strong>Category:</strong> ${safeText(book.category)}</p>
            <p>${safeText(book.description)}</p>
            <p class="${book.copies > 0 ? "available" : "unavailable"}">
                ${book.copies > 0
                    ? "Available Copies: " + book.copies
                    : "Currently Unavailable"}
            </p>
            <button onclick="viewBook(${book.id})">View Details</button>
        </div>
    `).join("");
}

function viewBook(id) {
    const book = books.find(item => item.id === id);
    if (!book) return;

    alert(
        "Book Details\n\n" +
        "Title: " + book.title + "\n" +
        "Author: " + book.author + "\n" +
        "Category: " + book.category + "\n" +
        "Description: " + book.description + "\n" +
      "Available Copies: " + book.copies
   );
}

// Librarian login
function adminLogin() {
    const email = document.getElementById("adminEmail").value.trim();
    const password = document.getElementById("adminPassword").value;

    // Demo credentials only
    if (email === "librarian@library.com" && password === "lib123") {
        isAdmin = true;

        document.getElementById("adminMessage").textContent =
            "Librarian login successful.";

        document.getElementById("adminDashboard").style.display = "block";
```

```
        displayAdminBooks();
        displayIssueOptions();
        displayAdminBorrowRecords();
        displayAdminMaterials();
        displayAdminStudents();
        displayAdminMessages();
    } else {
        document.getElementById("adminMessage").textContent =
            "Invalid librarian email or password.";
    }
}

// Add a book
function addBook() {
    if (!isAdmin) {
        alert("Please log in as librarian.");
        return;
    }

    const title = document.getElementById("newBookTitle").value.trim();
    const author = document.getElementById("newBookAuthor").value.trim();
    const category = document.getElementById("newBookCategory").value.trim();
    const copies = Number(document.getElementById("newBookCopies").value);
    const description = document.getElementById("newBookDescription").value.trim();

    if (!title || !author || !category ||
        !Number.isInteger(copies) || copies < 1 || !description) {
        alert("Please enter all book details and a valid number of copies.");
        return;
    }

    books.push({
        id: makeId(),
        title,
        author,
        category,
        copies,
        description
    });

    saveData();
    displayBooks();
    displayAdminBooks();
    displayIssueOptions();

    document.getElementById("newBookTitle").value = "";
    document.getElementById("newBookAuthor").value = "";
```

```
    document.getElementById("newBookCategory").value = "";
    document.getElementById("newBookCopies").value = "";
    document.getElementById("newBookDescription").value = "";

    alert("Book added successfully!");
}

// Admin book management
function displayAdminBooks() {
    if (!isAdmin) return;

    const list = document.getElementById("adminBookList");

    list.innerHTML = books.map(book => `
        <div class="admin-card">
            <p><strong>${safeText(book.title)}</strong></p>
            <p>Author: ${safeText(book.author)}</p>
            <p>Available copies: ${book.copies}</p>
            <button onclick="editBook(${book.id})">Update Book</button>
            <button onclick="deleteBook(${book.id})">Delete Book</button>
        </div>
    `).join("");
}

function editBook(id) {
    if (!isAdmin) return;

    const book = books.find(item => item.id === id);
    if (!book) return;

    const title = prompt("Enter book title:", book.title);
    if (title === null) return;

    const author = prompt("Enter author:", book.author);
    if (author === null) return;

    const category = prompt("Enter category:", book.category);
    if (category === null) return;

    const copiesInput = prompt("Enter total available copies:", book.copies);
    if (copiesInput === null) return;

    const copies = Number(copiesInput);
    const description = prompt("Enter description:", book.description);
    if (description === null) return;
```

```
    if (!title.trim() || !author.trim() || !category.trim() ||
        !Number.isInteger(copies) || copies < 0 || !description.trim()) {
        alert("Invalid book details.");
        return;
    }

    const activeIssues = borrowRecords.filter(
        record => record.bookId === id && record.status === "Issued"
    ).length;

    if (copies < activeIssues) {
        alert("Copies cannot be fewer than the books currently issued.");
        return;
    }

    book.title = title.trim();
    book.author = author.trim();
    book.category = category.trim();
    book.copies = copies;
    book.description = description.trim();

    saveData();
    displayBooks();
    displayAdminBooks();
    displayIssueOptions();
    displayHistory();
    displayAdminBorrowRecords();

    alert("Book updated successfully!");
}

function deleteBook(id) {
    if (!isAdmin) return;

    const activeIssue = borrowRecords.some(
        record => record.bookId === id && record.status === "Issued"
    );

    if (activeIssue) {
        alert("This book cannot be deleted while it has active issues.");
        return;
    }

    if (!confirm("Are you sure you want to delete this book?")) return;

    books = books.filter(book => book.id !== id);
```

```
    saveData();
    displayBooks();
    displayAdminBooks();
    displayIssueOptions();
}

// Book issue
function displayIssueOptions() {
    if (!isAdmin) return;

    const select = document.getElementById("issueBookId");

    select.innerHTML = books.map(book => `
        <option value="${book.id}" ${book.copies === 0 ? "disabled" : ""}>
            ${safeText(book.title)} (${book.copies} available)
        </option>
    `).join("");
}

function issueBook() {
    if (!isAdmin) return;

    const email = document.getElementById("issueEmail").value.trim().toLowerCase();
    const bookId = Number(document.getElementById("issueBookId").value);

    const student = students.find(item => item.email === email);
    const book = books.find(item => item.id === bookId);

    if (!student) {
        alert("Student not found. Ask the student to register first.");
        return;
    }

    if (!book || book.copies < 1) {
        alert("This book is not available.");
        return;
    }

    const alreadyIssued = borrowRecords.some(
        record => record.email === email &&
            record.bookId === bookId &&
            record.status === "Issued"
    );

    if (alreadyIssued) {
```

```
        alert("This student already has an active issue for this book.");
        return;
    }

    const issueDate = new Date();
    const expiryDate = new Date(issueDate);
    expiryDate.setDate(expiryDate.getDate() + 14);

    borrowRecords.push({
        id: makeId(),
        email,
        bookId,
        issueDate: issueDate.toISOString().slice(0, 10),
        expiryDate: expiryDate.toISOString().slice(0, 10),
        returnDate: "",
        status: "Issued",
        fine: 0
    });

    book.copies--;

    saveData();
    displayBooks();
    displayAdminBooks();
    displayIssueOptions();
    displayHistory();
    displayFine();
    displayAdminBorrowRecords();

    alert("Book issued successfully. Return it within 14 days.");
}

// Calculate a demo fine: ₹5 per overdue day
function calculateFine(record) {
    if (record.status !== "Issued" && record.status !== "Returned") {
        return 0;
    }

    const endDate = record.status === "Returned" && record.returnDate
        ? new Date(record.returnDate + "T00:00:00")
        : new Date();

    const dueDate = new Date(record.expiryDate + "T00:00:00");
    const lateDays = Math.max(
        0,
        Math.floor((endDate - dueDate) / (1000 * 60 * 60 * 24))
    );
```

```
    return lateDays * 5;
}

// Student borrowing history
function displayHistory() {
    const list = document.getElementById("historyList");

    if (!currentStudent) {
        list.innerHTML = "<p>Please log in to view your borrowing history.</p>";
        return;
    }

    const records = borrowRecords.filter(
        record => record.email === currentStudent.email
    );

    if (records.length === 0) {
        list.innerHTML = "<p>You have not borrowed any books yet.</p>";
        return;
    }

    list.innerHTML = records.slice().reverse().map(record => {
        const book = books.find(item => item.id === record.bookId);
        const fine = calculateFine(record);

        return `
            <div class="history-card">
                <h3>${safeText(book ? book.title : "Book record")}</h3>
                <p>Issue Date: ${safeText(record.issueDate)}</p>
                <p>Expected Return Date: ${safeText(record.expiryDate)}</p>
                <p>Actual Return Date: ${safeText(record.returnDate || "Not returned")}</p>
                <p>Status: <span class="status">${safeText(record.status)}</span></p>
                <p>Fine: ₹${fine}</p>
            </div>
        `;
    }).join("");
}

function displayFine() {
    const box = document.getElementById("fineDetails");

    if (!currentStudent) {
        box.textContent = "Please log in to check your fine.";
        return;
    }
```

```
    const records = borrowRecords.filter(
        record => record.email === currentStudent.email
    );

    const total = records.reduce(
        (sum, record) => sum + calculateFine(record), 0
    );

    box.innerHTML = `
        <p>Student: ${safeText(currentStudent.name)}</p>
        <p>Total outstanding / recorded fine: <strong>₹${total}</strong></p>
        <p>Demo fine rate: ₹5 per overdue day.</p>
    `;
}

// Admin return status and fine details
function displayAdminBorrowRecords() {
    if (!isAdmin) return;

    const list = document.getElementById("adminBorrowList");

    if (borrowRecords.length === 0) {
        list.innerHTML = "<p>No borrowing records available.</p>";
        return;
    }

    list.innerHTML = borrowRecords.slice().reverse().map(record => {
        const book = books.find(item => item.id === record.bookId);
        const fine = calculateFine(record);

        return `
            <div class="admin-card">
                <p><strong>Student:</strong> ${safeText(record.email)}</p>
                <p><strong>Book:</strong> ${safeText(book ? book.title : "Deleted book")}</p>
                <p>Issue Date: ${safeText(record.issueDate)}</p>
                <p>Due Date: ${safeText(record.expiryDate)}</p>
                <p>Status: ${safeText(record.status)}</p>
                <p>Calculated Fine: ₹${fine}</p>
                ${record.status === "Issued"
                    ? `<button onclick="returnBook(${record.id})">Mark as Returned</button>`
                    : ""}
            </div>
        `;
    }).join("");
}

function returnBook(id) {
```

```
    if (!isAdmin) return;

    const record = borrowRecords.find(item => item.id === id);

    if (!record || record.status !== "Issued") {
        alert("This book is already returned or the record is missing.");
        return;
    }

    const book = books.find(item => item.id === record.bookId);

    record.status = "Returned";
    record.returnDate = new Date().toISOString().slice(0, 10);

    if (book) book.copies++;

    record.fine = calculateFine(record);

    saveData();
    displayBooks();
    displayAdminBooks();
    displayIssueOptions();
    displayAdminBorrowRecords();
    displayHistory();
    displayFine();

    alert("Book returned successfully. Fine: ₹" + record.fine);
}

// Study materials
function displayMaterials() {
    const year = document.getElementById("materialYear").value;
    const semester = document.getElementById("materialSemester").value;

    const filtered = materials.filter(material =>
        (year === "All" || String(material.year) === year) &&
        (semester === "All" || String(material.semester) === semester)
    );

    const list = document.getElementById("materialList");

    if (filtered.length === 0) {
        list.innerHTML = "<p>No study materials found for this selection.</p>";
        return;
    }
```

```
    list.innerHTML = filtered.map(material => `
      <div class="material-card">
        <h3>${safeText(material.title)}</h3>
        <p>Subject: ${safeText(material.subject)}</p>
        <p>Year: ${material.year}</p>
        <p>Semester: ${material.semester}</p>
        <a href="${material.dataUrl}"
          download="${safeText(material.fileName)}"
          class="hero-btn">Download PDF</a>
      </div>
    `).join("");
}

// Upload small PDF files for this demo
async function uploadMaterial() {
    if (!isAdmin) {
      alert("Please log in as librarian.");
      return;
    }

    const title = document.getElementById("materialTitle").value.trim();
    const subject = document.getElementById("materialSubject").value.trim();
    const year = Number(document.getElementById("uploadYear").value);
    const semester = Number(document.getElementById("uploadSemester").value);
    const file = document.getElementById("materialFile").files[0];

    if (!title || !subject || !file) {
      alert("Enter the material title, subject and select a PDF.");
      return;
    }

    if (file.type !== "application/pdf" && !file.name.toLowerCase().endsWith(".pdf")) {
      alert("Please select a PDF file.");
      return;
    }

    if (file.size > 700 * 1024) {
      alert("For this browser demo, please use a PDF smaller than 700 KB.");
      return;
    }

    const reader = new FileReader();

    reader.onload = function () {
      materials.push({
        id: makeId(),
        title,
        subject,
```

```
            year,
            semester,
            fileName: file.name,
            dataUrl: reader.result
        });

        try {
            saveData();
        } catch (error) {
            materials.pop();
            alert("Storage is full. Try a smaller PDF or clear demo data.");
            return;
        }

        displayMaterials();
        displayAdminMaterials();

        document.getElementById("materialTitle").value = "";
        document.getElementById("materialSubject").value = "";
        document.getElementById("materialFile").value = "";

        alert("Study material added successfully!");
    };

    reader.onerror = function () {
        alert("Could not read this file.");
    };

    reader.readAsDataURL(file);
}

function displayAdminMaterials() {
    if (!isAdmin) return;

    const list = document.getElementById("adminMaterialList");

    if (materials.length === 0) {
        list.innerHTML = "<p>No study materials uploaded.</p>";
        return;
    }

    list.innerHTML = materials.map(material => `
        <div class="admin-card">
            <p><strong>${safeText(material.title)}</strong></p>
            <p>${safeText(material.subject)} — Year ${material.year}, Semester
${material.semester}</p>
            <button onclick="deleteMaterial(${material.id})">Delete Material</button>
```

```
        </div>
    `).join("");
}

function deleteMaterial(id) {
    if (!isAdmin) return;

    if (!confirm("Delete this study material?")) return;

    materials = materials.filter(material => material.id !== id);

    saveData();
    displayMaterials();
    displayAdminMaterials();
}

// Contact form
function sendMessage() {
    const name = document.getElementById("contactName").value.trim();
    const email = document.getElementById("contactEmail").value.trim();
    const message = document.getElementById("contactMessage").value.trim();

    if (!name || !email.includes("@") || !message) {
        alert("Enter your name, valid email and message.");
        return;
    }

    messages.push({
        id: makeId(),
        name,
        email,
        message,
        date: new Date().toLocaleDateString()
    });

    saveData();

    document.getElementById("contactResult").textContent =
        "Your message has been saved successfully.";

    document.getElementById("contactName").value = "";
    document.getElementById("contactEmail").value = "";
    document.getElementById("contactMessage").value = "";

    displayAdminMessages();
}
```

```
function displayAdminStudents() {
    if (!isAdmin) return;

    const list = document.getElementById("adminStudentList");

    if (students.length === 0) {
        list.innerHTML = "<p>No registered students.</p>";
        return;
    }

    list.innerHTML = students.map(student => `
        <div class="admin-card">
            <p><strong>${safeText(student.name)}</strong></p>
            <p>Email: ${safeText(student.email)}</p>
            <button onclick="deleteStudent(${student.id})">Delete Student</button>
        </div>
    `).join("");
}

function deleteStudent(id) {
    if (!isAdmin) return;

    const student = students.find(item => item.id === id);
    if (!student) return;

    const activeIssue = borrowRecords.some(
        record => record.email === student.email && record.status === "Issued"
    );

    if (activeIssue) {
        alert("Cannot delete a student who has books that have not been returned.");
        return;
    }

    if (!confirm("Delete this student account?")) return;

    students = students.filter(item => item.id !== id);

    saveData();
    displayAdminStudents();

    if (currentStudent && currentStudent.id === id) {
        logoutStudent();
    }
}
```

```
function displayAdminMessages() {
    if (!isAdmin) return;

    const list = document.getElementById("adminMessageList");

    if (messages.length === 0) {
        list.innerHTML = "<p>No contact messages received.</p>";
        return;
    }

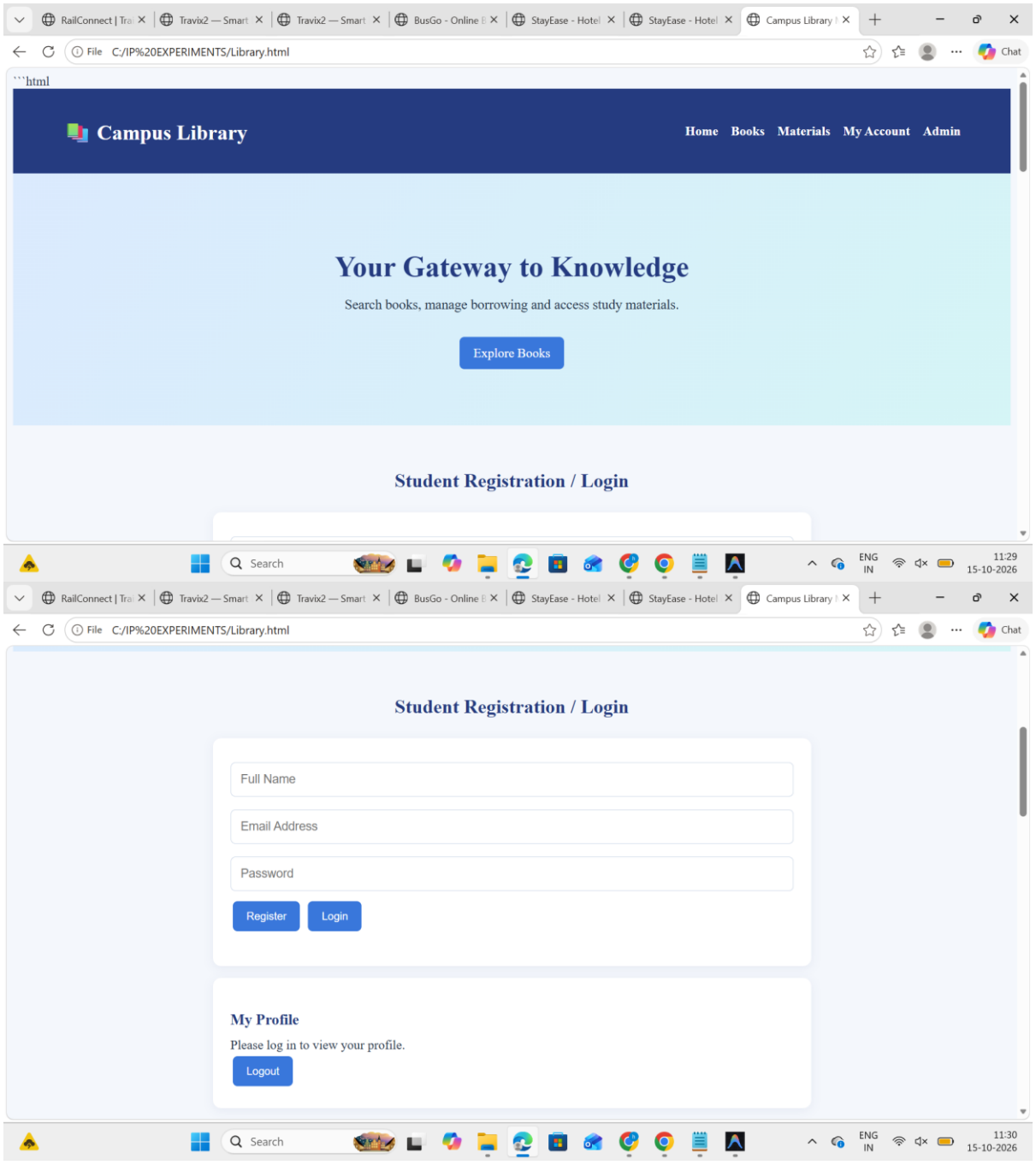
    list.innerHTML = messages.slice().reverse().map(message => `
        <div class="message-card">
            <p><strong>${safeText(message.name)}</strong></p>
            <p>Email: ${safeText(message.email)}</p>
            <p>Date: ${safeText(message.date)}</p>
            <p>${safeText(message.message)}</p>
        </div>
    `).join("");
}

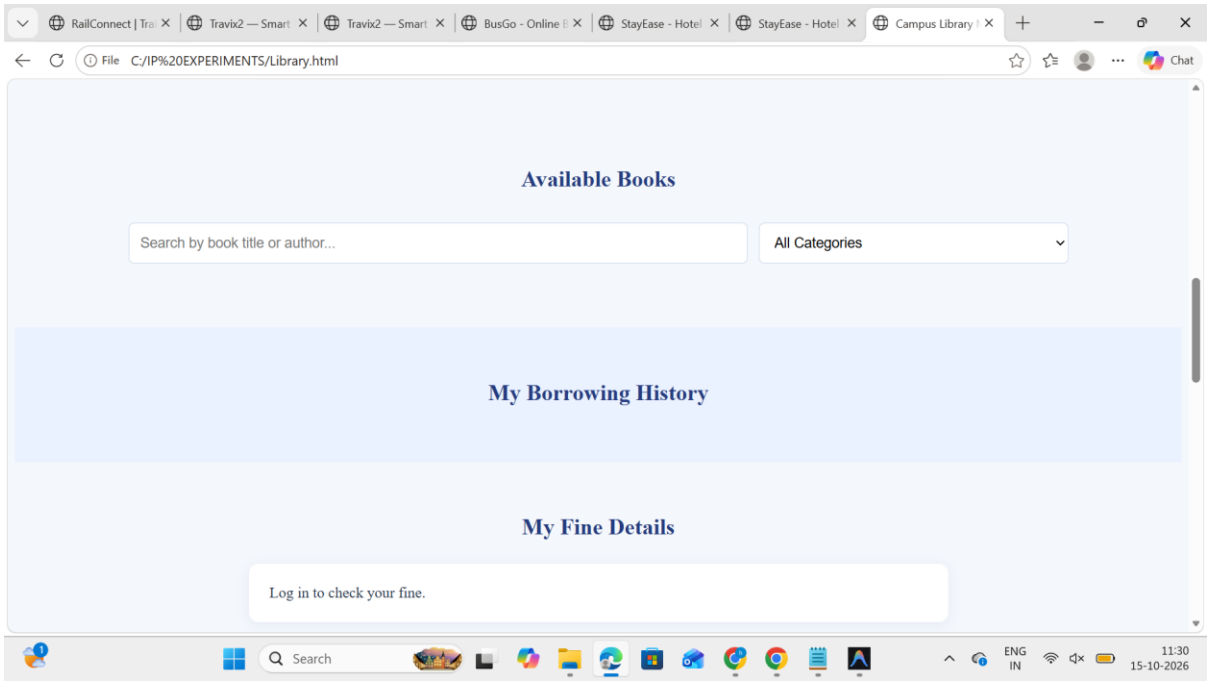
// Initial page display
displayBooks();
displayMaterials();
displayHistory();
displayFine();
```

Result

The library website was developed successfully. Students can search books, see issue and expiry dates, check fines and download study material, and the admin can manage books, issue and return books and calculate fines.

OUTPUT:


RailConnect | Tra
Travix2 — Smart
Travix2 — Smart
BusGo - Online B
StayEase - Hotel
StayEase - Hotel
Campus Library
File C:/IP%20EXPERIMENTS/Library.html
Chat
```html
Campus Library
Home Books Materials My Account Admin
Your Gateway to Knowledge
Search books, manage borrowing and access study materials.
Explore Books
Student Registration / Login
Search
ENG IN
11:29 15-10-2026
RailConnect | Tra
Travix2 — Smart
Travix2 — Smart
BusGo - Online B
StayEase - Hotel
StayEase - Hotel
Campus Library
File C:/IP%20EXPERIMENTS/Library.html
Chat
Student Registration / Login
Full Name
Email Address
Password
Register
Login
My Profile
Please log in to view your profile.
Logout
Search
ENG IN
11:30 15-10-2026
```


RailConnect | Tra
Travix2 — Smart
Travix2 — Smart
BusGo - Online B
StayEase - Hotel
StayEase - Hotel
Campus Library
File C:/IP%20EXPERIMENTS/Library.html
Chat
Available Books
Search by book title or author...
All Categories
My Borrowing History
My Fine Details
Log in to check your fine.
Search
ENG
IN
11:30
15-10-2026